RACIOCÍNIO AUTOMÁTICO PARA OPTIMIZAÇÃO

Luís Morgado

ISEL-DEI

PROCURA EM ESPAÇOS DE ESTADOS LOCAL

Métodos de *têmpera simulada* (*Simulated Annealing*)

- Possibilidade de exploração a partir do estado actual (exploração guiada)
  - **Amostragem local**
- Adaptação dinâmica de parâmetros do método de procura
- Restrição temporal
  - **Soluções aproximadas**
- Analogia com a têmpera do metal
  - **Aquecimento**
    - Flexibilidade
  - **Arrefecimento**
    - Restrição

PROCURA EM ESPAÇOS DE ESTADOS LOCAL

Algoritmo *Simulated Annealing*

Versão estocástica do algoritmo *Hill-Climbing*, no qual são possíveis transições para estados com menor valor que o estado actual

- Essas transições são aceites com uma probabilidade dependente de um parâmetro designado *temperatura*
  - Uma função de escalonamento determina a forma como a temperatura varia ao longo do tempo

function SIMULATED-ANNEALING($problem$, $schedule$) **returns** a solution state
  inputs: $problem$, a problem
    $schedule$, a mapping from time to "temperature"

  $current \leftarrow$ MAKE-NODE($problem$.INITIAL-STATE)
  for $t = 1$ **to** ∞ **do**
    $T \leftarrow schedule(t)$
    if $T = 0$ **then return** $current$
    $next \leftarrow$ a randomly selected successor of $current$
    $\Delta E \leftarrow next$.VALUE – $current$.VALUE
    if $\Delta E > 0$ **then** $current \leftarrow next$
    else $current \leftarrow next$ only with probability $e^{\Delta E/T}$

[Russel & Norvig, 2010]

PROCURA EM ESPAÇOS DE ESTADOS LOCAL

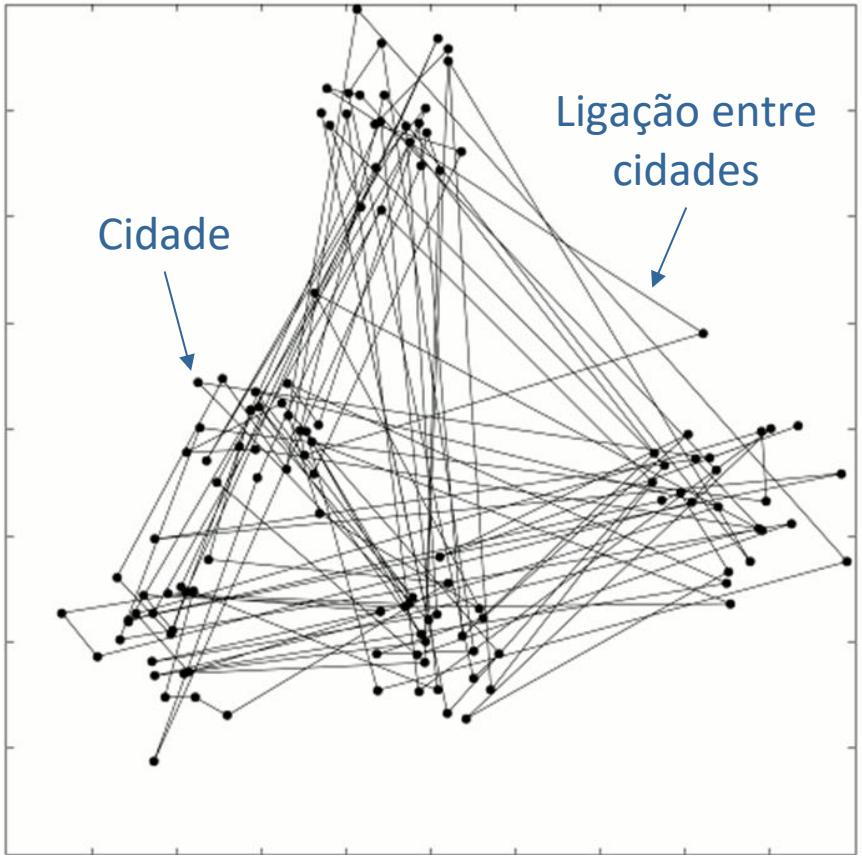
Algoritmo *Simulated Annealing* detalhado

```
1. function simulated_annealing(problema, f_T, t_max)
2.     estado = problema.estado_inicial
3.     valor_estado = problema.avaliar(estado)
4.     for t = 0 to t_max:
5.         T = f_T(t)
6.         sucessores = problema.sucessores(estado)
7.         sucessor = random.choice(sucessores)
8.         if problema.solucao(sucessor):
9.             estado = sucessor
10.            break
11.        valor_sucessor = problema.avaliar(sucessor)
12.        ΔE = valor_sucessor - valor_estado
13.        if ΔE > 0 or random() < e^(ΔE/T):
14.            estado = sucessor
15.            valor_estado = valor_sucessor
16.    return estado
```

ALGORITMO *SIMULATED ANNEALING*

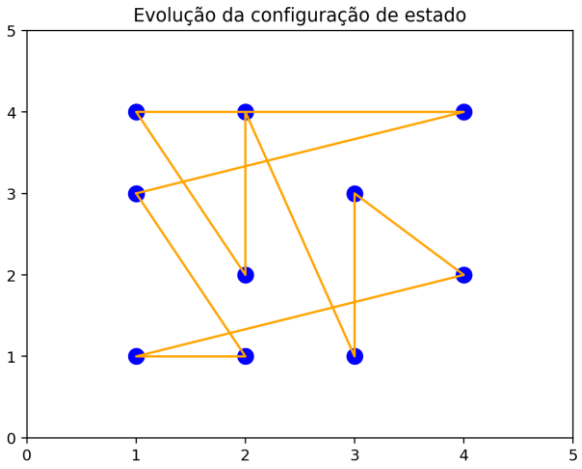
Exemplo: Problema do caixeiro viajante

Determinar o menor percurso para visitar um conjunto de cidades, retornando à cidade de origem

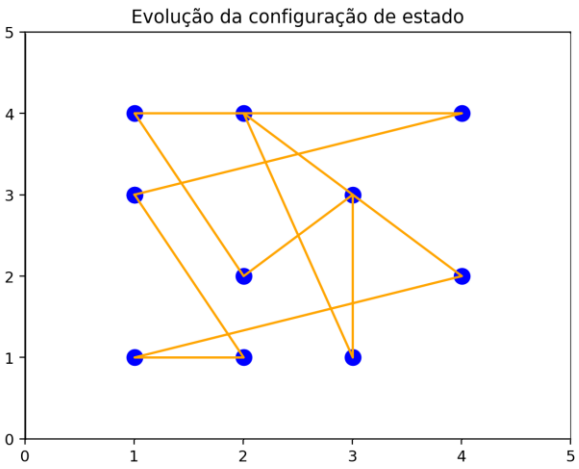


Exemplo: Problema do caixeiro viajante

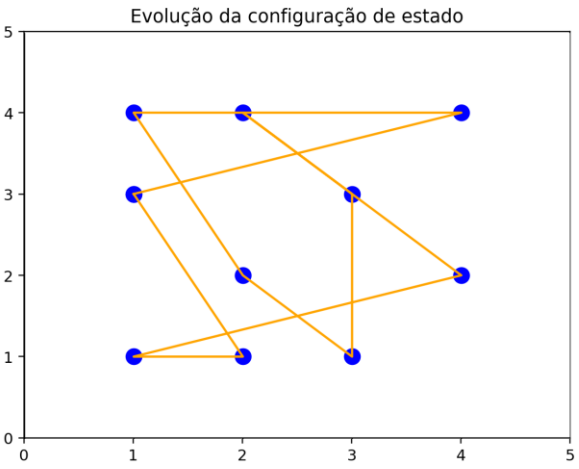
$t = 0$



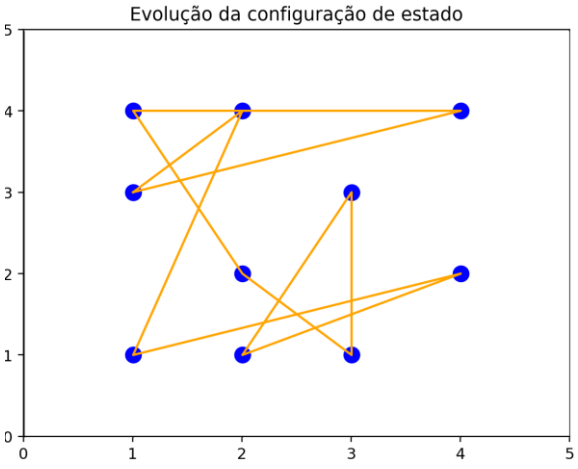
$t = 1$



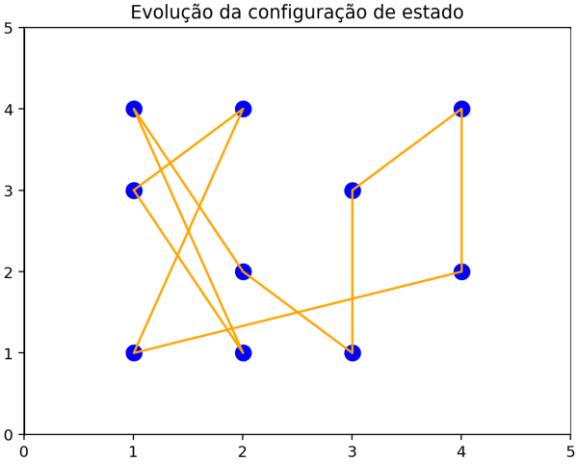
$t = 2$



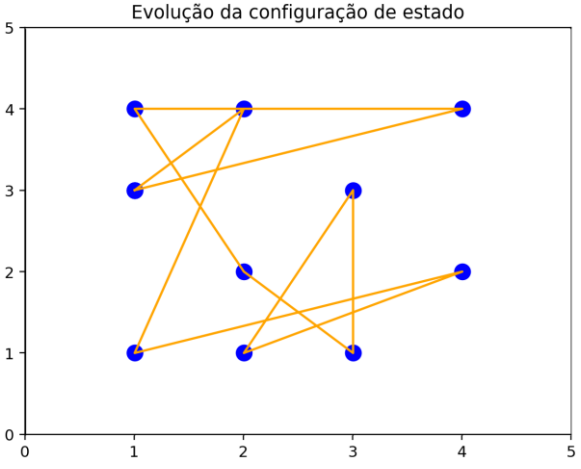
$t = 3$



$t = 4$

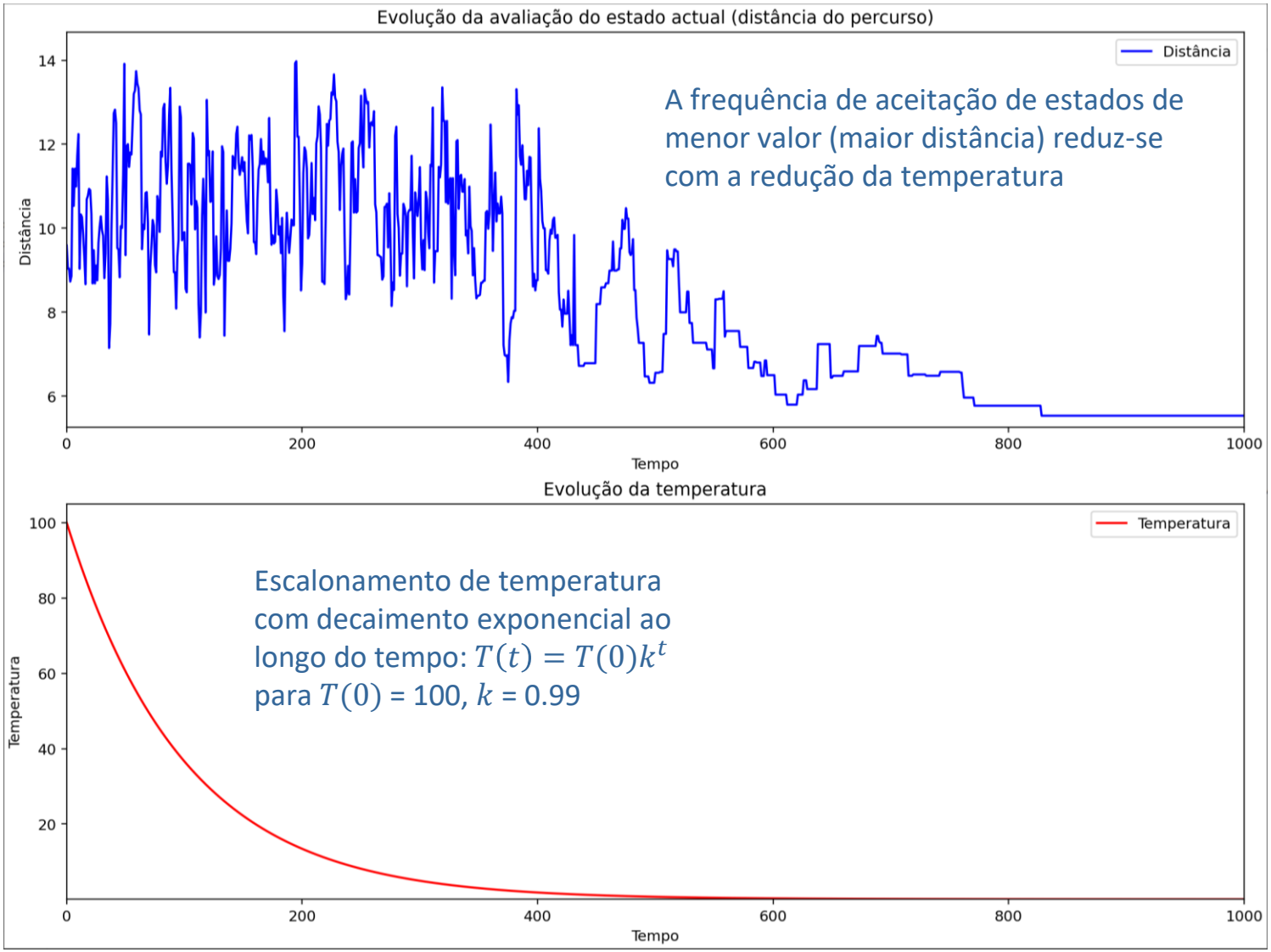


$t = 5$



Dimensão do problema ***n* = 10** (número de cidades)

Exemplo: Problema do caixeiro viajante



Dimensão do problema ***n* = 10** (número de cidades)

PROCURA EM ESPAÇOS DE ESTADOS LOCAL

Elementos a considerar na resolução de um problema de optimização

- **Problema**
  - Representa informação de contexto do problema
  - Função de avaliação de estado
    - Por exemplo, distâncias entre cidades no problema do caixeiro viajante
  - Estado inicial
  - Transição de estado
    - Conjunto de operadores ou função de geração de estados sucessores

PROCURA EM ESPAÇOS DE ESTADOS LOCAL

- **Estado**
  - Representa uma configuração de resolução do problema
    - Por exemplo, configuração de rainhas no tabuleiro no problema das N-Rainhas
  - Realiza encapsulamento de manipulação da configuração de estado
  - A sua representação deve expressar as restrições que caracterizam o problema

PROCURA EM ESPAÇOS DE ESTADOS LOCAL

- **Transição de estado**
  - Para um determinado estado, produz:
    - Conjunto de estados sucessores
  - Deve produzir transformações locais de estado, reflectindo as características do domínio do problema
    - Por exemplo:
      - Movimentação de uma rainha no problema das N-Rainhas
      - Alteração de sequência parcial do percurso no problema do caixeiro viajante
  - **Transformações de estado com carácter aleatório correspondem à introdução de um pendor de amostragem global**
    - Degeneram numa procura puramente estocástica
    - Degradação de desempenho

PROCURA EM ESPAÇOS DE ESTADOS LOCAL

- **Resultados**
  - Solução do problema
    - Pode não ser a solução óptima
  - Qualidade da solução
    - Em função da qualidade da solução encontrada, esta pode ser aceite ou serem explorados outros métodos de optimização
  - Número de iterações (tempo) para obter a solução
    - Indicação do esforço computacional para produzir a solução

REFERÊNCIAS

[Russel & Norvig, 2010]

S. Russell, P. Norvig, “Artificial Intelligence: A Modern Approach”, 3rd Ed., Prentice Hall, 2010

[Beire *et. Al.*, 2014]

A. Beire, N. Cota, H. Pita, A. Rodrigues, “Automatic tuning of Okumura-Hata model on railway communications”, International Symposium on Wireless Personal Multimedia Communications (WPMC), 2014

[Nagy *et al.*, 2018]

D. Nagy, D. Zhao, D. Benjamin, “Nature-Based Hybrid Computational Geometry System for Optimizing Component Structure”, Humanizing Digital Reality, Springer Nature, 2018